Basit Multiplayer Simülasyon

Bu bölümde, öğrendiğimiz tüm kavramları (Client-Server mimarisi, TCP/UDP, Serialization) birleştirerek basit ama çalışan bir multiplayer simülasyonu gerçekleştireceğiz. İki istemcinin, bir sunucu üzerinden konumlarını senkronize ettiği (position sync) bir sistem kuracağız. Bu uygulama, oyunlardaki "netcode" mantığının temelini anlamanızı sağlayacaktır.

---

1. Game Loop Mantığı (Oyun Döngüsü)

Bir oyunun kalbi, sürekli çalışan ve oyun durumunu güncelleyen "Game Loop" (Oyun Döngüsü)'dür. Bu döngü, oyunun her karesinde (frame) belirli işlemleri tekrarlar.

1.1. Temel Game Loop Bileşenleri

```
while (oyun çalışıyorken) {
    1. Input Al (Kullanıcı girişlerini oku)
    2. Update (Oyun mantığını güncelle – fizik, yapay zeka, çarpışma)
    3. Render (Güncel durumu ekrana çiz)
}
```

1.2. Single Player vs Multiplayer Game Loop

Single Player	Multiplayer (Client-Server)
Tüm işlemler (Input, Update, Render) aynı cihazda, sırayla yapılır.	**Sunucu** kendi game loop'unu çalıştırır (Update). **İstemci** hem kendi game loop'unu çalıştırır (Render + Input) hem de sunucudan gelen güncellemeleri işler.
Update ve Render arasında senkronizasyon kolaydır.	Sunucu ve istemci farklı hızlarda çalışabilir. Senkronizasyon için "Tick" sistemi kullanılır.
Gecikme (latency) sorunu yoktur.	Gecikme, en büyük zorluktur ve telafi mekanizmaları (Lag Compensation) gerektirir.

1.3. Multiplayer Game Loop Akışı (İstemci Perspektifi)

```
while (oyun çalışıyorken) {
    // 1. Input Toplama
    inputs = getPlayerInputs();

    // 2. Input'u Sunucuya Gönder (C2S)
    client.send(inputs);

    // 3. Sunucudan Gelen Güncellemeleri Al ve İşle (S2C)
    if (data = client.receive()) {
        updateLocalWorld(data); // Sunucunun gönderdiği dünya durumunu
uygula
    }
```

```
    // 4. Yerel Tahmin (Local Prediction - İleride detaylandırılacak)
    // Oyuncunun kendi hareketini, sunucu yanıtı gelmeden tahmin et

    // 5. Render (Ekrana Çiz)
    render();
}
```

---

2. Tick Rate Nedir? (Sunucu Güncelleme Sıklığı)

Tick rate, sunucunun oyun dünyasını güncelleme sıklığıdır. Saniyede kaç kez sunucu, oyun durumunu (fizik, konum, çarpışma) hesaplar ve istemcilere güncelleme gönderir.

2.1. Temel Tanımlar

- **Tick (Sunucu Adımı):** Sunucu game loop'unun bir iterasyonu.
- **Tick Rate (Tick Hızı):** Saniyedeki tick sayısı. Örneğin `tickrate = 60` demek, sunucunun saniyede 60 kez güncellendiği anlamına gelir.
- **Tick Interval (Tick Aralığı):** Her tick arasındaki süre. `tickrate = 60` ise, `interval = 1000ms / 60 ≈ 16.67ms`.

2.2. Oyunlarda Yaygın Tick Rate Değerleri

Oyun / Sistem	Tick Rate	Açıklama
Counter-Strike: GO (Resmi)	64	Standart rekabetçi oyun sunucuları.
Counter-Strike: GO (Faceit/ESEA)	128	Profesyonel liglerde kullanılır, daha hassas atış ve hareket.
Valorant	128	Yüksek tick rate, rekabetçi oyun için optimize.
Battlefield / COD	60-120	Hızlı aksiyon oyunları, yüksek tick rate tercih edilir.
MMORPG (WoW)	20-30	Düşük tick rate, çünkü anlık refleksler kritik değil.
Oyun Sunucusu (Genel Öneri)	20-60	Oyun türüne göre değişir. FPS için yüksek, strateji için orta.

2.3. Tick Rate ve Bant Genişliği İlişkisi

Yüksek tick rate, daha duyarlı (responsive) bir oyun deneyimi sunar, ancak daha fazla bant genişliği tüketir. Örneğin:

- Bir oyuncu konumu için binary veri = 20 byte.
- 60 oyuncu varsa ve tickrate = 60 ise: `20 byte * 60 oyuncu * 60 tick = 72,000 byte/s ≈ 70 KB/s` (sadece konum verisi).

Bu nedenle, oyun geliştiricileri tick rate ve bant genişliği arasında bir denge kurmalıdır.

---

3. Uygulama: 2 Client → 1 Server (Basit Position Sync)

Şimdi, iki istemcinin konumlarını bir sunucu üzerinden senkronize ettiği basit bir sistem kuracağız.

Mimari:

1. **Sunucu (Server):** Her istemciden gelen konum verilerini alır, diğer istemciye iletir (broadcast).
2. **İstemci 1 (Client 1):** Klavye girdileriyle (WASD) konumunu değiştirir, bu konumu sunucuya gönderir ve diğer istemcinin konumunu sunucudan alır.
3. **İstemci 2 (Client 2):** Client 1 ile aynı mantıkta çalışır.

Kullanılacak Protokol: UDP (hızlı, anlık güncellemeler için)

Kullanılacak Veri Formatı: JSON (okunabilirlik ve debug kolaylığı için)

3.1. Sunucu Kodu (server.py)

```
import socket
import json
import time

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

class GameServer:
    def __init__(self):
        self.server_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.server_socket.bind((HOST, PORT))
        self.clients = {}  # {addr: {"id": int, "x": float, "y": float,
"last_update": time}}
        self.next_player_id = 1
        self.tick_rate = 20  # Saniyede 20 tick
        self.tick_interval = 1.0 / self.tick_rate
        print(f"Sunucu {HOST}:{PORT} adresinde çalışıyor... (Tick Rate:
{self.tick_rate})")

    def run(self):
        """Ana sunucu döngüsü."""
        last_tick_time = time.time()

        while True:
            # Her tick'te yapılacak işlemler
            current_time = time.time()
            if current_time - last_tick_time >= self.tick_interval:
                self.tick()
                last_tick_time = current_time

            # Non-blocking mesaj alma (timeout ile)
            self.server_socket.settimeout(0.01)  # 10ms timeout
            try:
```

```
                data, addr = self.server_socket.recvfrom(BUFFER_SIZE)
                self.handle_message(data, addr, current_time)
            except socket.timeout:
                continue
            except Exception as e:
                print(f"Hata: {e}")

    def handle_message(self, data, addr, current_time):
        """Gelen mesajı işler."""
        try:
            message = json.loads(data.decode('utf-8'))
            msg_type = message.get('type')

            if msg_type == 'JOIN':
                # Yeni oyuncu katılımı
                player_id = self.next_player_id
                self.next_player_id += 1
                self.clients[addr] = {
                    "id": player_id,
                    "x": 0.0,
                    "y": 0.0,
                    "last_update": current_time
                }
                print(f"Oyuncu {player_id} katıldı! (Address: {addr})")

                # Yeni oyuncuya kendi ID'sini gönder
                response = {
                    "type": "ASSIGN_ID",
                    "playerId": player_id,
                    "tickRate": self.tick_rate
                }
                self.server_socket.sendto(json.dumps(response).encode('utf-
8'), addr)

                # Diğer oyunculara yeni oyuncuyu bildir
                self.broadcast_players(addr)

            elif msg_type == 'POSITION_UPDATE':
                # Konum güncellemesi
                if addr in self.clients:
                    player_id = self.clients[addr]["id"]
                    self.clients[addr]["x"] = message.get('x', 0.0)
                    self.clients[addr]["y"] = message.get('y', 0.0)
                    self.clients[addr]["last_update"] = current_time

            elif msg_type == 'DISCONNECT':
                # Oyuncu ayrılıyor
                if addr in self.clients:
                    player_id = self.clients[addr]["id"]
                    print(f"Oyuncu {player_id} ayrıldı.")
                    del self.clients[addr]
                    self.broadcast_players(addr)

        except json.JSONDecodeError:
```

```
            print(f"Geçersiz JSON: {data}")
        except Exception as e:
            print(f"Mesaj işleme hatası: {e}")

    def broadcast_players(self, exclude_addr=None):
        """Tüm oyuncuların durumunu istemcilere gönderir."""
        players_list = []
        for addr, player_data in self.clients.items():
            players_list.append({
                "id": player_data["id"],
                "x": player_data["x"],
                "y": player_data["y"]
            })

        response = {
            "type": "STATE_UPDATE",
            "players": players_list,
            "tick": int(time.time() * 1000)  # Sunucu zamanı
        }

        response_bytes = json.dumps(response).encode('utf-8')

        for addr in self.clients.keys():
            if addr != exclude_addr:
                self.server_socket.sendto(response_bytes, addr)

    def tick(self):
        """Her tick'te yapılacak işlemler."""
        # Bağlantısı kesilen oyuncuları kontrol et (30 saniye
güncellenmemişse)
        current_time = time.time()
        timeout_seconds = 30.0
        disconnected = []

        for addr, player_data in self.clients.items():
            if current_time - player_data["last_update"] > timeout_seconds:
                disconnected.append(addr)
                print(f"Oyuncu {player_data['id']} zaman aşımına uğradı.")

        for addr in disconnected:
            del self.clients[addr]

        # Eğer bir oyuncu ayrıldıysa durumu güncelle
        if disconnected:
            self.broadcast_players(None)

        # Tüm istemcilere düzenli olarak durum gönder
        if len(self.clients) > 0:
            self.broadcast_players(None)

if __name__ == "__main__":
    server = GameServer()
    try:
        server.run()
```

```
    except KeyboardInterrupt:
        print("\nSunucu kapatılıyor...")
        server.server_socket.close()
```

3.2. İstemci Kodu (client.py)

```
import socket
import json
import threading
import time
import keyboard  # Klavye girdileri için (pip install keyboard)

HOST = '127.0.0.1'
PORT = 12345
BUFFER_SIZE = 1024

class GameClient:
    def __init__(self):
        self.client_socket = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
        self.player_id = None
        self.local_x = 0.0
        self.local_y = 0.0
        self.other_players = {}  # {player_id: {"x": float, "y": float}}
        self.running = True
        self.tick_rate = 20
        self.tick_interval = 1.0 / self.tick_rate

        # Sunucuya bağlan
        self.connect_to_server()

    def connect_to_server(self):
        """Sunucuya JOIN mesajı gönderir ve ID bekler."""
        join_message = {"type": "JOIN"}
        self.client_socket.sendto(json.dumps(join_message).encode('utf-8'),
(HOST, PORT))
        print("Sunucuya bağlanılıyor...")

        # ID almak için bekle
        self.client_socket.settimeout(3.0)
        try:
            data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
            response = json.loads(data.decode('utf-8'))
            if response.get('type') == 'ASSIGN_ID':
                self.player_id = response['playerId']
                self.tick_rate = response.get('tickRate', 20)
                self.tick_interval = 1.0 / self.tick_rate
                print(f"Oyuncu ID: {self.player_id} atandı. (Tick Rate:
{self.tick_rate})")
            else:
                print("Beklenmeyen yanıt:", response)
```

```
                self.running = False
            except socket.timeout:
                print("Sunucudan yanıt alınamadı!")
                self.running = False
            except Exception as e:
                print(f"Bağlantı hatası: {e}")
                self.running = False

        def send_position(self):
            """Mevcut konumu sunucuya gönderir."""
            if self.player_id is None:
                return

            position_message = {
                "type": "POSITION_UPDATE",
                "x": self.local_x,
                "y": self.local_y
            }

            try:
                self.client_socket.sendto(
                    json.dumps(position_message).encode('utf-8'),
                    (HOST, PORT)
                )
            except Exception as e:
                print(f"Konum gönderme hatası: {e}")

        def receive_messages(self):
            """Sunucudan gelen mesajları sürekli dinler."""
            self.client_socket.settimeout(0.05)
            while self.running:
                try:
                    data, _ = self.client_socket.recvfrom(BUFFER_SIZE)
                    message = json.loads(data.decode('utf-8'))
                    self.handle_server_message(message)
                except socket.timeout:
                    continue
                except json.JSONDecodeError:
                    continue
                except Exception as e:
                    print(f"Alım hatası: {e}")
                    break

        def handle_server_message(self, message):
            """Sunucudan gelen mesajı işler."""
            msg_type = message.get('type')

            if msg_type == 'STATE_UPDATE':
                # Diğer oyuncuların durumlarını güncelle
                players = message.get('players', [])
                self.other_players.clear()
                for player in players:
                    if player['id'] != self.player_id:
                        self.other_players[player['id']] = {
```

```
                        'x': player['x'],
                        'y': player['y']
                    }

            # Debug: Konsola yazdır
            self.debug_print_state()

        elif msg_type == 'ASSIGN_ID':
            # Yeniden ID ataması (genelde ilk bağlantıda olur)
            self.player_id = message['playerId']
            print(f"Oyuncu ID güncellendi: {self.player_id}")

    def debug_print_state(self):
        """Mevcut oyun durumunu konsola yazdırır."""
        # Terminal ekranını temizle
        print("\033c", end="")
        print(f"=== Oyuncu {self.player_id} ===")
        print(f"Ben: X={self.local_x:.1f}, Y={self.local_y:.1f}")
        print("\n--- Diğer Oyuncular ---")
        if not self.other_players:
            print("Başka oyuncu yok.")
        else:
            for pid, pos in self.other_players.items():
                print(f"Oyuncu {pid}: X={pos['x']:.1f}, Y={pos['y']:.1f}")
        print("\n[WASD] Hareket | [Q] Çıkış")

    def handle_input(self):
        """Klavye girdilerini işler."""
        move_speed = 3.0
        last_send_time = 0

        while self.running:
            current_time = time.time()

            # Hareket inputları
            moved = False
            if keyboard.is_pressed('w'):
                self.local_y += move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('s'):
                self.local_y -= move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('a'):
                self.local_x -= move_speed * self.tick_interval
                moved = True
            if keyboard.is_pressed('d'):
                self.local_x += move_speed * self.tick_interval
                moved = True

            # Q ile çıkış
            if keyboard.is_pressed('q'):
                print("Çıkılıyor...")
                self.disconnect()
                break
```

```
            # Konumu tick rate'de gönder
            if moved or (current_time - last_send_time >=
self.tick_interval):
                self.send_position()
                last_send_time = current_time

            # CPU tüketimini azalt
            time.sleep(0.01)

    def disconnect(self):
        """Sunucudan ayrılma mesajı gönderir ve bağlantıyı kapatır."""
        self.running = False
        if self.player_id is not None:
            disconnect_message = {"type": "DISCONNECT"}
            try:
                self.client_socket.sendto(
                    json.dumps(disconnect_message).encode('utf-8'),
                    (HOST, PORT)
                )
            except:
                pass
        self.client_socket.close()
        print("Bağlantı kapatıldı.")

    def run(self):
        """İstemciyi çalıştırır."""
        if not self.running:
            return

        print("İstemci başlatıldı. WASD ile hareket edin.")
        print("Q tuşuna basarak çıkabilirsiniz.")

        # Mesaj alma thread'i
        receive_thread = threading.Thread(target=self.receive_messages)
        receive_thread.daemon = True
        receive_thread.start()

        # Input işleme (ana thread)
        try:
            self.handle_input()
        except KeyboardInterrupt:
            print("\nKullanıcı tarafından durduruldu.")
        finally:
            self.disconnect()

if __name__ == "__main__":
    client = GameClient()
    client.run()
```

---

4. Sistemin Çalışma Mantığı

1. **Sunucu Başlatılır:** Sunucu, UDP üzerinden mesajları dinlemeye başlar. Tick rate = 20 (saniyede 20 güncelleme).
2. **İstemci Başlatılır:** İstemci sunucuya `JOIN` mesajı gönderir.
3. **ID Atanır:** Sunucu, istemciye `ASSIGN_ID` mesajı ile benzersiz bir oyuncu ID'si gönderir.
4. **İlk Durum Paylaşımı:** Sunucu, bağlı olan tüm oyuncuların durumunu (konum) istemcilere `STATE_UPDATE` olarak gönderir.
5. **Hareket Döngüsü:**
    - İstemci WASD tuşlarına basıldığında `local_x` ve `local_y` değerlerini günceller.
    - İstemci, her tick aralığında (veya hareket olduğunda) konumunu `POSITION_UPDATE` ile sunucuya gönderir.
    - Sunucu, gelen konum verilerini kendi hafızasında günceller.
    - Sunucu, her tick'te (saniyede 20 kez) tüm istemcilere güncel oyuncu listesini `STATE_UPDATE` olarak gönderir.
6. **Görselleştirme:** İstemci, sunucudan gelen `STATE_UPDATE` mesajları ile diğer oyuncuların konumlarını `other_players` sözlüğünde günceller ve ekrana yazdırır.

---

5. Karşılaşılabilecek Sorunlar ve Çözüm Önerileri

Sorun	Nedeni	Çözüm
Oyuncu "sıçramalı" (teleporting) ilerliyor.	İstemci konumu yeterince sık göndermiyor.	Tick rate'i artırın (örneğin 30'a).
Diğer oyuncu hareketleri geç geliyor.	Sunucu güncellemeleri yeterince sık gönderilmiyor.	Sunucu tick rate'ini artırın.
Paket kaybı nedeniyle konumlar senkron değil.	UDP kullanıldığı için bazı paketler düşüyor.	İstemci, her tick'te konumu yeniden göndersin (aşırı yük değilse).
Oyuncu bağlantısı koptu ama listede gözüküyor.	Sunucu zaman aşımı (timeout) kontrolü yapmıyor.	Sunucuda `last_update` kontrolü ekledik (30 saniye).
Konsol çok hızlı güncelleniyor, okunmuyor.	`STATE_UPDATE` çok sık geliyor.	Debug çıktısını her güncellemede değil, saniyede 1-2 kez yazdırın.

---

6. Geliştirme Önerileri

1. **Grafik Arayüz (GUI) Ekleme:** Pygame veya Tkinter kullanarak oyuncuları daireler veya kareler olarak ekranda gösterin.
2. **Renk Farklılaştırması:** Her oyuncuyu farklı renkte gösterin.
3. **Oturum Açma / Kapatma Mesajları:** Oyuncu katıldığında/ayrıldığında ekrana bilgi yazdırın.
4. **Hız Ayarı:** `move_speed` değerini oyuncudan oyuncuya farklı yapın (örneğin "hızlı" ve "yavaş" karakterler).
5. **Mesafe Hesaplama:** Diğer oyunculara olan mesafeyi hesaplayıp ekrana yazdırın.

- **Game Loop:** Oyunun sürekli çalışan kalbi. Multiplayer'da sunucu ve istemci ayrı ayrı game loop çalıştırır.

- **Tick Rate:** Sunucunun güncelleme sıklığı. Yüksek tick rate daha duyarlı ancak daha fazla bant genişliği tüketir.
- **Position Sync:** İstemciler konumlarını sunucuya gönderir (C2S), sunucu tüm istemcilere güncel durumu dağıtır (S2C). Bu, multiplayer oyunların en temel veri alışverişidir.
- **UDP Tercihi:** Hızlı ve anlık güncellemeler için UDP idealdir. Paket kayıpları tolere edilebilir.
- **JSON ile Prototipleme:** Geliştirme aşamasında JSON kullanmak, debug ve protokol geliştirmeyi kolaylaştırır.

Bu simülasyon, gerçek bir oyunun network altyapısının en temel versiyonudur. Buradan yola çıkarak, lag compensation, client-side prediction, server reconciliation gibi ileri konulara geçiş yapabilirsiniz.